NumPy : Numeric Python

In [177…
```
pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\mania\anaconda3\lib\site-package
s (2.2.2)Note: you may need to restart the kernel to use updated packages.
```

In [178…
```
import numpy as np
```

Arrays

In [182…
```
arr1 = np.array([10,20,30,40,50])
```

In [184…
```
arr1
```

Out[184…
```
array([10, 20, 30, 40, 50])
```

In [186…
```
print(type(arr1))
```

```
<class 'numpy.ndarray'>
```

In [188…
```
np.array(range(1,11))
```

Out[188…
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [190…
```
arr2 = np.array([[1,2],[3,4]])
```

In [192…
```
arr2.shape
```

Out[192…
```
(2, 2)
```

In [194…
```
arr3 = np.array([[[1,2,0],[3,4,0]], [[5,6,0],[7,8,0]]])
```

In [196…
```
arr3
```

Out[196…
```
array([[[1, 2, 0],
        [3, 4, 0]],

       [[5, 6, 0],
        [7, 8, 0]]])
```

In [198…
```
arr3.shape
```

Out[198…
```
(2, 2, 3)
```

In [200…
```
arr = np.array([1,2.5,3], dtype=int)
```

In [202…
```
arr
```

Out[202…
```
array([1, 2, 3])
```

In [204…
```
arr = np.array([{1:"one"},15])
```

In [206…
```
arr
```

Out[206…
```
array([{1: 'one'}, 15], dtype=object)
```

In [208…
```
arr1
```

Out[208…
```
array([10, 20, 30, 40, 50])
```

In [210…
```
arr1[0] = 0
```

In [212…
```
arr1
```

Out[212…
```
array([ 0, 20, 30, 40, 50])
```

In [214…
```
arr1.ctypes.data
```

Out[214…
```
2133146775904
```

In [216…
```
arr1.itemsize
```

Out[216…
```
8
```

In [218…
```
for i in range(len(arr1)):
    print(arr1.ctypes.data + i*arr1.itemsize)
```

```
2133146775904
2133146775912
2133146775920
2133146775928
2133146775936
```

Array Attributes

In [221…
```
arr1.size
```

Out[221…
```
5
```

In [223…
```
arr1.ndim
```

Out[223…
```
1
```

In [225…
```
arr1.shape
```

Out[225…
```
(5,)
```

In [227…
```
arr1.itemsize
```

Out[227…
```
8
```

In [229…
```
arr1.nbytes
```

Out[229…
```
40
```

In [231…
```
arr1.ctypes.data
```

Out[231… 2133146775904

Array Creation Routines

In [234…
```
np.zeros(5, dtype = int)
```

Out[234…
```
array([0, 0, 0, 0, 0])
```

In [236…
```
np.zeros((2,3))
```

Out[236…
```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

In [238…
```
np.ones(4)
```

Out[238…
```
array([1., 1., 1., 1.])
```

In [240…
```
np.ones((3,5), dtype = str)
```

Out[240…
```
array([['1', '1', '1', '1', '1'],
       ['1', '1', '1', '1', '1'],
       ['1', '1', '1', '1', '1']], dtype='<U1')
```

In [242…
```
x = np.full((2,3), 3)
```

In [244…
```
x
```

Out[244…
```
array([[3, 3, 3],
       [3, 3, 3]])
```

In [246…
```
np.empty((1,3,4))
```

Out[246…
```
array([[[1.05391271e-311, 2.47032823e-322, 0.00000000e+000,
         0.00000000e+000],
        [1.10343781e-312, 1.31370903e-076, 5.27727559e-091,
         9.96444437e-047],
        [3.60966526e+174, 2.44258303e-052, 3.99910963e+252,
         1.33978813e-075]]])
```

In [248…
```
np.arange(1,10,3)
```

Out[248…
```
array([1, 4, 7])
```

In [250…
```
np.arange(9).reshape(3,3)
```

Out[250…
```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

In [252…
```
np.linspace(1,10,5)
```

Out[252…
```
array([ 1.  ,  3.25,  5.5 ,  7.75, 10.  ])
```

In [254…
```
np.linspace(0,100,6)
```

Out[254…
```
array([  0.,  20.,  40.,  60.,  80., 100.])
```

In [256…
```
arr = np.arange(1,10).reshape(3,3)
```

In [258…
```
arr
```

Out[258…
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

In [260…
```
arr.T
```

Out[260…
```
array([[1, 4, 7],
       [2, 5, 8],
       [3, 6, 9]])
```

In [262…
```
np.eye(3)
```

Out[262…
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

In [264…
```
np.eye(5,3)
```

Out[264…
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 0., 0.],
       [0., 0., 0.]])
```

In [266…
```
np.identity(3)
```

Out[266…
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

In [268…
```
np.identity(5,3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[268], line 1
----> 1 np.identity(5,3)

File ~\anaconda3\Lib\site-packages\numpy\_core\numeric.py:2233, in identity(n, dtype, like)
   2230     return _identity_with_like(like, n, dtype=dtype)
   2232 from numpy import eye
-> 2233 return eye(n, dtype=dtype, like=like)

File ~\anaconda3\Lib\site-packages\numpy\lib\_twodim_base_impl.py:222, in eye(N, M, k, dtype, order, device, like)
    220 if M is None:
    221     M = N
--> 222 m = zeros((N, M), dtype=dtype, order=order, device=device)
    223 if k >= M:
    224     return m

TypeError: Cannot interpret '3' as a data type
```

Random number generation

randint()

In [272…
```
np.random.randint(0,40, (2,3))
```

Out[272…
```
array([[11,  1, 35],
       [ 4, 25,  7]], dtype=int32)
```

rand()

In [275…
```
np.random.rand() #0 <= x < 1
```

Out[275…
```
0.5715088421402174
```

In [277…
```
np.random.rand(5)
```

Out[277…
```
array([0.89581164, 0.40845842, 0.55416618, 0.95367279, 0.76291103])
```

In [279…
```
np.random.rand(2,3)
```

Out[279…
```
array([[0.9661913 , 0.81598183, 0.30025089],
       [0.7425056 , 0.01612781, 0.89019419]])
```

randn() - normal distribution

In [282…
```
np.random.randn() # -3 <= x <= 3
```

Out[282…
```
0.9810825431690297
```

In [284…
```
np.random.rand(10) * 25
```

Out[284…
```
array([12.21516326, 14.88928799, 23.89543559, 16.32800476, 19.59483818,
        5.61001846, 15.47567263, 23.91791048, 17.09493549,  7.49082853])
```

In [286…
```
np.random.randn(10)*5 + 25
```

Out[286…
```
array([25.40878804, 21.21399795, 11.59503938, 22.09074608, 17.56226265,
       28.09668058, 25.82733889, 27.67663096, 19.80705066, 26.2555884 ])
```

choice

In [289…
```
roll_no = list(range(101,111))
```

In [291…
```
roll_no
```

Out[291…
```
[101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
```

In [293…
```
print(np.random.choice(roll_no, 3, replace = False))
```

```
[109 103 106]
```

shuffle

In [296…
```
arr = np.arange(9)
```

In [298…
```
arr
```

Out[298…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8])
```

In [300…
```
np.random.shuffle(arr)
```

In [302…
```
arr
```

Out[302…
```
array([1, 0, 6, 7, 5, 2, 8, 4, 3])
```

permutation

In [305…
```
arr = np.arange(9)
```

In [307…
```
arr
```

Out[307…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8])
```

In [309…
```
np.random.permutation(arr)
```

Out[309…
```
array([0, 8, 2, 7, 6, 3, 1, 4, 5])
```

In [311…
```
arr
```

Out[311…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8])
```

seed

In [314…
```
np.random.seed(78)
np.random.randint(1,100)
```

Out[314…
```
70
```

uniform

In [317…
```
np.random.uniform(1,5,3)
```

Out[317…
```
array([4.5493321 , 3.29343038, 3.38632804])
```

In [319…
```
np.random.randint(5)
```

Out[319…
```
4
```

normal

In [322…
```
np.random.seed(0)
np.random.normal(25,5,10) #(mean, std, no_of_elements)
```

Out[322…
```
array([33.82026173, 27.00078604, 29.89368992, 36.204466  , 34.33778995,
       20.1136106 , 29.75044209, 24.24321396, 24.48390574, 27.05299251])
```

In [324…
```
np.random.randn(10)*5 + 25
```

Out[324…
```
array([25.72021786, 32.27136753, 28.80518863, 25.60837508, 27.21931616,
       26.66837164, 32.47039537, 23.97420868, 26.56533851, 20.7295213 ])
```

Aggregation Functions

sum(), mean(), max(), min(), argmax(), argmin(),

median(), std(), var(), prod(), cumsum(), cumprod()

In [337…
```
arr1 = np.array(range(1,11))
```

In [339…
```
arr1
```

Out[339…
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [341…
```
arr1.sum()
```

Out[341…
```
np.int64(55)
```

In [343…
```
np.sum(arr1)
```

Out[343…
```
np.int64(55)
```

In [345…
```
arr2 = np.array(range(1,10)).reshape(3,3)
```

In [347…
```
arr2
```

Out[347…
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

In [349…
```
np.sum(arr2, axis = 0)
```

Out[349…
```
array([12, 15, 18])
```

In [351…
```
arr2.mean(axis = 1)
```

Out[351…
```
array([2., 5., 8.])
```

In [353…
```
arr1.min()
```

Out[353…
```
np.int64(1)
```

In [355…
```
arr2.max()
```

Out[355…
```
np.int64(9)
```

In [357…
```
arr1.argmin()
```

Out[357…
```
np.int64(0)
```

In [359…
```
arr1.argmax()
```

Out[359… np.int64(9)

In [361…
```
arr1.median()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[361], line 1
----> 1 arr1.median()

AttributeError: 'numpy.ndarray' object has no attribute 'median'
```

In [363…
```
np.median(arr1)
```

Out[363… np.float64(5.5)

In [365…
```
np.std(arr1)
```

Out[365… np.float64(2.8722813232690143)

In [367…
```
np.var(arr1)
```

Out[367… np.float64(8.25)

In [369…
```
np.prod(arr1)
```

Out[369… np.int64(3628800)

In [371…
```
np.prod(arr2, axis = 0)
```

Out[371…
```
array([ 28,  80, 162])
```

In [373…
```
arr1
```

Out[373…
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [375…
```
np.cumsum(arr1)
```

Out[375…
```
array([ 1,  3,  6, 10, 15, 21, 28, 36, 45, 55])
```

In [377…
```
np.cumprod(arr1)
```

Out[377…
```
array([      1,       2,       6,      24,     120,     720,    5040,
         40320,  362880, 3628800])
```

Indexing & Slicing

In [380…
```
arr1 = np.array(range(2,21,2))
```

In [382…
```
arr1
```

Out[382…
```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

In [384…
```
arr1[5]
```

Out[384… np.int64(12)

In [386…
```
arr1[8]
```

Out[386…
```
np.int64(18)
```

In [388…
```
arr1[:5]
```

Out[388…
```
array([ 2,  4,  6,  8, 10])
```

In [390…
```
arr1[4:]
```

Out[390…
```
array([10, 12, 14, 16, 18, 20])
```

In [392…
```
arr1[-4:]
```

Out[392…
```
array([14, 16, 18, 20])
```

In [394…
```
arr1[::2]
```

Out[394…
```
array([ 2,  6, 10, 14, 18])
```

In [396…
```
arr1[::-1]
```

Out[396…
```
array([20, 18, 16, 14, 12, 10,  8,  6,  4,  2])
```

In [398…
```
arr2 = np.array([[2,4,6,8],[1,3,5,7],[11,22,33,44],[10,20,30,40]])
```

In [400…
```
arr2
```

Out[400…
```
array([[ 2,  4,  6,  8],
       [ 1,  3,  5,  7],
       [11, 22, 33, 44],
       [10, 20, 30, 40]])
```

In [402…
```
arr2[-1]
```

Out[402…
```
array([10, 20, 30, 40])
```

In [404…
```
arr2
```

Out[404…
```
array([[ 2,  4,  6,  8],
       [ 1,  3,  5,  7],
       [11, 22, 33, 44],
       [10, 20, 30, 40]])
```

In [406…
```
arr2[-2:]
```

Out[406…
```
array([[11, 22, 33, 44],
       [10, 20, 30, 40]])
```

In [408…
```
arr2[::2,:]
```

Out[408…
```
array([[ 2,  4,  6,  8],
       [11, 22, 33, 44]])
```

In [410…
```
arr2
```

Out[410…
```
array([[ 2,  4,  6,  8],
       [ 1,  3,  5,  7],
       [11, 22, 33, 44],
       [10, 20, 30, 40]])
```

In [412…
```
arr2[:,:2]
```

Out[412…
```
array([[ 2,  4],
       [ 1,  3],
       [11, 22],
       [10, 20]])
```

In [414…
```
arr2[:2,:2]
```

Out[414…
```
array([[2, 4],
       [1, 3]])
```

In [416…
```
arr2[1:3,1:3]
```

Out[416…
```
array([[ 3,  5],
       [22, 33]])
```

In [418…
```
arr2[1:3,:3:2]
```

Out[418…
```
array([[ 1,  5],
       [11, 33]])
```

In [420…
```
arr2[:,::-1]
```

Out[420…
```
array([[ 8,  6,  4,  2],
       [ 7,  5,  3,  1],
       [44, 33, 22, 11],
       [40, 30, 20, 10]])
```

In [422…
```
arr2[::-1,:]
```

Out[422…
```
array([[10, 20, 30, 40],
       [11, 22, 33, 44],
       [ 1,  3,  5,  7],
       [ 2,  4,  6,  8]])
```

In [424…
```
arr2[::-1, ::-1]
```

Out[424…
```
array([[40, 30, 20, 10],
       [44, 33, 22, 11],
       [ 7,  5,  3,  1],
       [ 8,  6,  4,  2]])
```

Arithmetic Operations

In [427…
```
a1 = np.array(range(1,6))
```

In [429…
```
a2 = np.arange(10,51,10)
```

In [431…
```
a1
```

Out[431…
```
array([1, 2, 3, 4, 5])
```

In [433…
```
a2
```

Out[433…
```
array([10, 20, 30, 40, 50])
```

In [435…
```
a1 + a2
```

Out[435…
```
array([11, 22, 33, 44, 55])
```

In [437…
```
a1 - a2
```

Out[437…
```
array([ -9, -18, -27, -36, -45])
```

In [439…
```
a1 / a2
```

Out[439…
```
array([0.1, 0.1, 0.1, 0.1, 0.1])
```

In [441…
```
a1 * a2
```

Out[441…
```
array([ 10,  40,  90, 160, 250])
```

In [443…
```
a1 // a2
```

Out[443…
```
array([0, 0, 0, 0, 0])
```

In [445…
```
a1 ** a2
```

Out[445…
```
array([                   1,              1048576,       205891132094649,
                          0, -6776596920136667815])
```

In [447…
```
ar1 = np.random.randint(10,50,9).reshape(3,3)
```

In [449…
```
ar2 = np.ones((3,3),dtype = int)
```

In [451…
```
ar2
```

Out[451…
```
array([[1, 1, 1],
       [1, 1, 1],
       [1, 1, 1]])
```

In [453…
```
ar1
```

Out[453…
```
array([[10, 10, 46],
       [15, 48, 27],
       [25, 14, 41]], dtype=int32)
```

In [455…
```
ar1 + ar2
```

Out[455…
```
array([[11, 11, 47],
       [16, 49, 28],
       [26, 15, 42]])
```

In [457…
```
ar1 - ar2
```

Out[457…
```
array([[ 9,  9, 45],
       [14, 47, 26],
       [24, 13, 40]])
```

In [459…
```
ar1 / ar2
```

Out[459…
```
array([[10., 10., 46.],
       [15., 48., 27.],
       [25., 14., 41.]])
```

In [461…
```
ar1 // ar2
```

Out[461…
```
array([[10, 10, 46],
       [15, 48, 27],
       [25, 14, 41]])
```

In [463…
```
ar1 % ar2
```

Out[463…
```
array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

In [465…
```
ar1 ** ar2
```

Out[465…
```
array([[10, 10, 46],
       [15, 48, 27],
       [25, 14, 41]])
```

In [467…
```
ar1 * ar2
```

Out[467…
```
array([[10, 10, 46],
       [15, 48, 27],
       [25, 14, 41]])
```

In [469…
```
ar1 @ ar2
```

Out[469…
```
array([[66, 66, 66],
       [90, 90, 90],
       [80, 80, 80]])
```

In [471…
```
np.matmul(ar1,ar2)
```

Out[471…
```
array([[66, 66, 66],
       [90, 90, 90],
       [80, 80, 80]])
```

In [473…
```
l1 = ['1','2','3']
```

In [475…
```
l1 * 3
```

Out[475…
```
['1', '2', '3', '1', '2', '3', '1', '2', '3']
```

In [477…
```
a1
```

Out[477…
```
array([1, 2, 3, 4, 5])
```

In [479…
```
a1 * 3
```

Out[479…
```
array([ 3,  6,  9, 12, 15])
```

In [481…
```
l1 = [1,2,3]
```

In [483…
```
[int(i)*3 for i in l1]
```

Out[483…
```
[3, 6, 9]
```

In [499…
```
a1 = np.array([40,50,20,10,30])
```

In [501…
```
a1
```

Out[501…
```
array([40, 50, 20, 10, 30])
```

In [503…
```
np.sort(a1)
```

Out[503…
```
array([10, 20, 30, 40, 50])
```

In [505…
```
a1
```

Out[505…
```
array([40, 50, 20, 10, 30])
```

In [507…
```
a1.sort()
```

In [509…
```
a1
```

Out[509…
```
array([10, 20, 30, 40, 50])
```

In [511…
```
a1 = np.array([40,50,20,10,30])
```

In [513…
```
np.sort(a1)[::-1]
```

Out[513…
```
array([50, 40, 30, 20, 10])
```

In [515…
```
a1
```

Out[515…
```
array([40, 50, 20, 10, 30])
```

In [517…
```
a1.sort()
```

In [519…
```
a1 = a1[::-1]
```

In [521…
```
a1
```

Out[521…
```
array([50, 40, 30, 20, 10])
```

In [523…
```
ar1
```

Out[523…
```
array([[10, 10, 46],
       [15, 48, 27],
       [25, 14, 41]], dtype=int32)
```

In [525…
```
np.sort(ar1, axis = 0)
```

Out[525…
```
array([[10, 10, 27],
       [15, 14, 41],
       [25, 48, 46]], dtype=int32)
```

In [527…
```
np.sort(ar1, axis = 1)
```

Out[527…
```
array([[10, 10, 46],
       [15, 27, 48],
       [14, 25, 41]], dtype=int32)
```

In [529…
```
a1 = np.array([40,50,30,20,10])
```

In [531…
```
np.argsort(a1)
```

Out[531…
```
array([4, 3, 2, 0, 1])
```

Conditional Selection

In [534…
```
marks = np.random.randint(25,30,10)
```

In [536…
```
marks
```

Out[536…
```
array([25, 26, 26, 26, 26, 28, 28, 27, 28, 25], dtype=int32)
```

In [538…
```
marks > 27
```

Out[538…
```
array([False, False, False, False, False,  True,  True, False,  True,
       False])
```

In [540…
```
marks[marks>27]
```

Out[540…
```
array([28, 28, 28], dtype=int32)
```

In [542…
```
marks[marks % 2 == 0]
```

Out[542…
```
array([26, 26, 26, 26, 28, 28, 28], dtype=int32)
```

In [544…
```
marks[(marks > 28) & (marks < 30)]
```

Out[544…
```
array([], dtype=int32)
```

In [546…
```
a1
```

Out[546…
```
array([40, 50, 30, 20, 10])
```

In [548…
```
a1[(a1%3 != 0) | (a1 > 30)]
```

Out[548…
```
array([40, 50, 20, 10])
```

In [550…
```
a1[~(a1%3!=0)]
```

Out[550…
```
array([30])
```

In [552…
```
ar1
```

Out[552…
```
array([[10, 10, 46],
       [15, 48, 27],
       [25, 14, 41]], dtype=int32)
```

In [554…
```
ar1[ar1>35]
```

Out[554… array([46, 48, 41], dtype=int32)